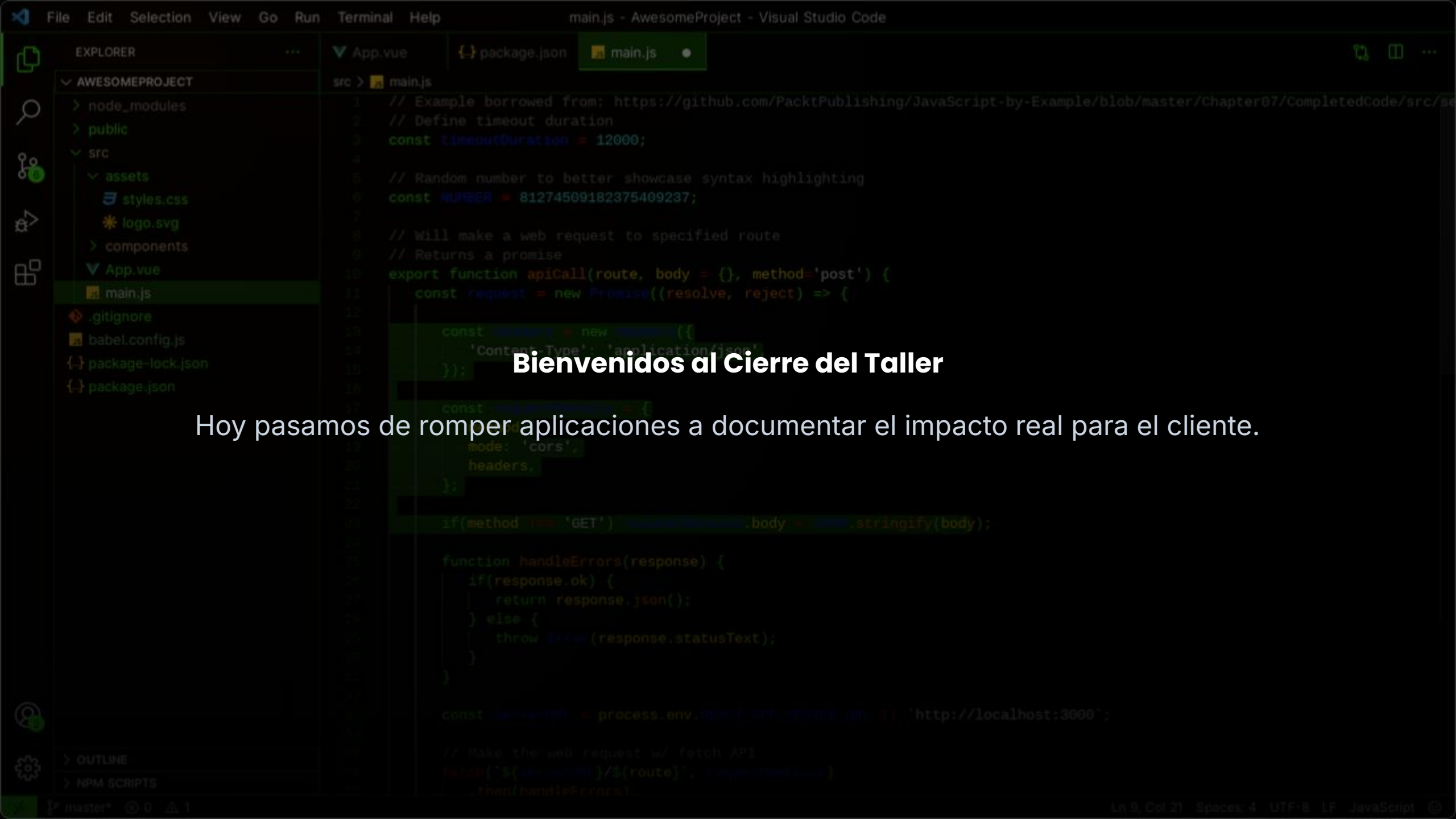


Hacking Web

SESIÓN 10: THE DARK SIDE

Deserialización Insegura & El Arte del Reporte Profesional



Bienvenidos al Cierre del Taller

Hoy pasamos de romper aplicaciones a documentar el impacto real para el cliente.

| Agenda del Día



Deserialización

Teoría técnica de objetos, formatos de datos y el riesgo del "vudú" en memoria.



Labs Prácticos

Manipulación de atributos y tipos de datos en entornos controlados de Burp Suite.



Pentest Reporting

Estructura de entregables, severidad CVSS y cómo transformar bugs en valor.

BLOQUE 1: DESERIALIZACIÓN

| ¿Qué es la Serialización?

Estado a String

Es el proceso de convertir un objeto complejo (con sus atributos y valores) en un formato plano (string o binario) para ser transportado o guardado.

Uso Común



Guardar sesiones en bases de datos.



Pasar objetos entre microservicios.



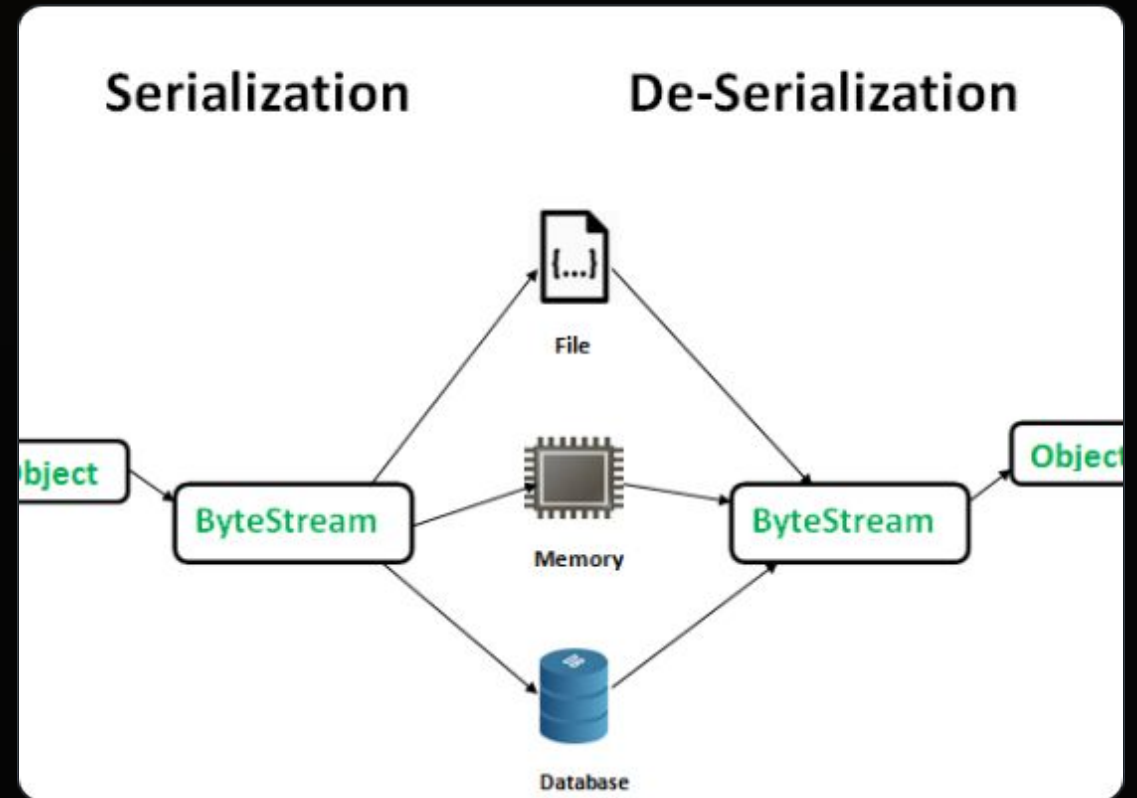
Almacenar estados en cookies de cliente.

| ¿Qué es la Deserialización?

Es el proceso inverso: el servidor toma el string y "reconstruye" el objeto original en la memoria.

```
$user = unserialize($_COOKIE['data']);
```

EL PELIGRO: Si el servidor confía ciegamente en el input del usuario para reconstruir este objeto, el atacante puede inyectar datos maliciosos.



| ¿Por qué es el "Lado Oscuro"?

- ⚡ **RCE (Remote Code Execution):** Es el impacto más común al encadenar gadgets.
- 👤 **Privilege Escalation:** Modificación de roles (admin:true).
- 📁➔ **Arbitrary File Read:** Acceso a archivos del sistema mediante objetos de flujo.

Es difícil de detectar con scanners automáticos porque requiere lógica del lenguaje.



| Anatomía de un Objeto PHP

```
O:4:"User":2:{s:8:"username";s:5:"admin";s:7:"isAdmin";b:0;}
```

O:4:"User"

Objeto de la clase "User" (4 caracteres).

s:8:"username"

String de 8 caracteres con valor "username".

b:0

Boolean con valor falso (0).

| Magic Methods: Los disparadores

`__wakeup()`

Se ejecuta inmediatamente cuando un objeto es deserializado. Ideal para inicializar conexiones o verificar estados.

`__destruct()`

Se ejecuta cuando el script termina o el objeto se destruye. Es un vector común para borrar archivos o cerrar logs.

Estos métodos permiten al atacante ejecutar código sin llamar explícitamente a una función.

| Vector 1: Atributos Manipulados

Es la forma más simple de explotación. El atacante intercepta el string serializado (ej: en una cookie) y cambia los valores.

Original:

```
0:4:"User":1:{s:5:"admin";b:0;}
```

Ataque:

```
0:4:"User":1:{s:5:"admin";b:1;}
```

Escalación Directa

Al procesar la cookie, el servidor cree que el usuario actual tiene privilegios administrativos.

| Vector 2: Confusión de Tipos

Muchos lenguajes (como PHP) usan "loose comparison" (==). Si comparamos un string con un integer 0, PHP puede dar true.

```
if ($password == 0) { // Bypass if password starts with non-numeric }
```

En el objeto serializado, podemos cambiar un **s:20:"password_complejo"** por un **i:0** para intentar engañar a la lógica de autenticación.

| Lab 01: Modifying Objects

El Escenario

Una aplicación web usa serialización de objetos PHP para manejar sesiones en una cookie. El objeto contiene un atributo de acceso.

La Misión

Identificar la cookie serializada, decodificarla (Base64/URL), cambiar el atributo de acceso y re-enviarla para ganar privilegios.

| Lab 01: Paso a Paso

-  Capturar petición en Burp Proxy.
-  Enviar a 'Decoder' y decodificar.
-  Cambiar b:0 por b:1.
-  Re-codificar y enviar en el Repeater.

```
s:12:"publicString";s:11:"Hello World";  
Int";i:42;s:11:"publicFloat";d:3.14159;  
b:1;s:10:"publicNull";N;  
;a:3:{i:0;s:5:"apple";i:1;s:6:"banana";  
s:18:"\NUL*\NULprotectedString";  
ta";s:15:"\NUL*\NULprotectedInt";i:100;  
ctedFloat";d:2.71828;  
ctedBool";b:0;  
ctedArray";a:3:{s:4:"name";s:8:"John Doe";  
0;s:6:"skills";a:3:{i:0;s:3:"PHP";i:1;  
i:2;s:6:"Python";  
l\NULprivateString";s:14:"Private Data";  
riginal\NULprivateInt";i:999;  
ULprivateFloat";d:1.41421;  
ULprivateBool";b:1;  
ULprivateArray";a:1:{s:6:"config";  
:1;s:7:"version";s:5:"1.0.0";
```

PHP Serialized Object viewer

☒ Monitor Clipboard

PHP serialized data detected and converted

Clipboard Content

```
{  
  "__classname": "Original",  
  "publicString": "Hello World",  
  "publicInt": 42,  
  "publicFloat": 3.14159,  
  "publicBool": true,  
  "publicNull": null,  
  "publicArray": [  
    "apple",  
    "banana",  
    "cherry"  
  ],  
  "protectedString": "Protected Data",  
  "protectedInt": 100,  
  "protectedFloat": 2.71828,  
  "protectedBool": false,  
  "protectedArray": {  
    "name": "John Doe",  
    "age": 30,  
    "skills": [  
      "PHP",  
      "Python"  
    ]  
  }  
}
```

| Lab 02: Data Type Tampering

El Desafío de Lógica

La aplicación compara el token de acceso usando el operador `==` de PHP. Si el token esperado es un string que no empieza por número, compararlo con el entero 0 resultará en true.

Objetivo: Modificar el tipo de dato del token de String (s) a Integer (i) con valor 0.

| Lab 02: Ejecución

```
// Antes  
s:32:"a1b2c3d4...";  
  
// Después (Ataque)  
i:0;
```

¿Por qué funciona?

PHP intenta convertir el string secreto a un número para compararlo con el 0 que enviamos. Como el secreto empieza con letras, PHP lo evalúa como 0.

0 == 0 → TRUE

| POP Chains: Encadenando Gadgets

A veces no basta con cambiar un atributo. Necesitamos que el servidor ejecute código.

Gadget

Un pedazo de código existente en la aplicación que puede ser usado para un fin no intencionado.

Cadena

Encadenar varios gadgets (métodos mágicos) para saltar de uno a otro hasta llegar al RCE.

Herramientas como **PHPGGC** o **ysoserial** automatizan la creación de estas cadenas.

| ¿Cómo nos protegemos?



No deserializar

Evitar pasar objetos serializados desde fuentes no confiables (el usuario).

Usar JSON/XML

Preferir formatos de datos planos que no reconstruyen objetos complejos.



Firma Digital

Si es obligatorio usarlo, firmar el string (HMAC) para verificar que no fue alterado.

| Tu Arsenal de Deserialización

PHPGGC

Generador de Gadget Chains para PHP. Imprescindible para RCE.

Ysoserial

La herramienta de referencia para deserialización en Java.

Burp Suite

Extensiones como 'Java Deserialization Scanner' o 'Taint Analysis'.

BLOQUE 2:

PENTESTING REPORTING

| El Reporte: Tu Producto Final

Un pentest no termina cuando consigues la shell. Termina cuando el cliente entiende el riesgo y cómo arreglarlo.

- 👁️ Visibilidad del riesgo.
- 🛡️ Guía de remediación.
- 💼 Justificación de presupuesto.



| Audiencia 1: El Resumen Ejecutivo

¿Para quién es?

CEOs, Gerentes, Directores. Gente que toma decisiones financieras y de riesgo.

¿Qué debe contener?

-  Postura general de seguridad.
-  Impacto de negocio (Pérdida de datos, multas).
-  Resumen de hallazgos críticos.

Regla de oro: Cero tecnicismos en esta sección.

| Audiencia 2: Detalles Técnicos

¿Para quién es?

Desarrolladores, Sysadmins, Equipos de Seguridad.
La gente que va a "picar código".

Contenido Vital



Descripción de la vulnerabilidad.



Evidencia y screenshots claros.



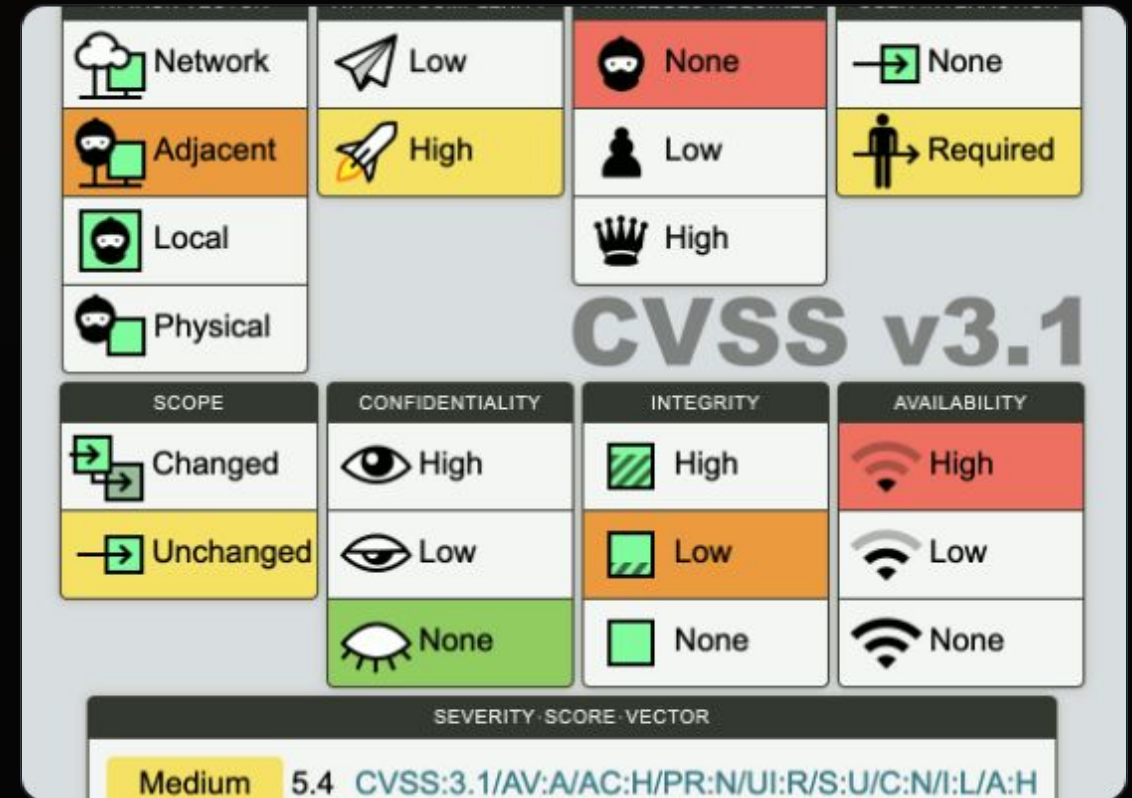
Pasos para reproducir (PoC).

| Severidad: El estándar CVSS

Common Vulnerability Scoring System (CVSS) ayuda a priorizar.

- **Bajo (0.1 - 3.9)**
- **Medio (4.0 - 6.9)**
- **Alto (7.0 - 8.9)**
- **Crítico (9.0 - 10.0)**

Factores: Vector de ataque, complejidad, privilegios, impacto en CIA.



| CVSS 3.1: ¿CÓMO MEDIR EL RIESGO?

El **Common Vulnerability Scoring System** es el estándar para calcular la severidad de forma objetiva.

Métrica	Valores Típicos
Vector de Ataque (AV)	Network (N), Adjacent (A), Local (L), Physical (P)
Complejidad (AC)	Low (L), High (H)
Privilegios Req. (PR)	None (N), Low (L), High (H)
Impacto (C, I, A)	None (N), Low (L), High (H)

| DESGLOSE DE MÉTRICAS BASE

Interacción (UI)

¿Requiere que la víctima haga clic? En Deserialización suele ser **None (N)**.

Alcance (S)

¿Afecta a otros sistemas? **Unchanged (U)** vs **Changed (C)**.

Triada CIA

Confidencialidad, Integridad y Disponibilidad. Lo que realmente se rompe.

| EJEMPLO CVSS: LAB 01

Escenario: Escalación de Privilegios

Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Puntuación: 8.8 (ALTO)

Ataque por red, complejidad baja, privilegios de usuario, impacto total en CIA.

| ANATOMÍA DE UN HALLAZGO

-  **Título:** Claro y directo.
-  **Severidad:** CVSS v3.1 Score.
-  **Descripción:** ¿Qué está pasando técnicamente?
-  **Impacto:** ¿Qué pierde el negocio?
-  **PoC:** Pasos para reproducir.
-  **Remediación:** ¿Cómo se soluciona?

| EJEMPLO DE REPORTE (TÍTULO & SEVERIDAD)

Vulnerabilidad de Deserialización Insegura en Cookie de Sesión

Severidad: 8.8 Crítico (CVSS v3.1)

Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

| EJEMPLO DE REPORTE (DESCRIPCIÓN & IMPACTO)

Descripción

La aplicación confía en un objeto serializado almacenado en la cookie session. Un atacante puede manipular los atributos del objeto antes de ser procesado por la función unserialize().

Impacto

Permite a un usuario autenticado escalar privilegios a administrador, comprometiendo la integridad y confidencialidad total de la base de datos de usuarios.

| EJEMPLO DE REPORTE (POC)

Prueba de Concepto (PoC)

- 1 Capturar la petición HTTP GET /admin.
- 2 Extraer el valor Base64 de la cookie 'session'.
- 3 Decodificar y modificar el atributo isAdmin de 0 a 1.
- 4 Re-codificar y enviar la petición modificada.
- 5 Verificar acceso exitoso al panel administrativo.

.

| EJEMPLO DE REPORTE (REMEDIACIÓN)

Solución Recomendada

Sustitución de Formato: Reemplazar la serialización nativa de objetos por formatos de intercambio de datos planos como JSON (`json_encode/decode`).

Integridad: Si es imperativo usar serialización, implementar una firma HMAC para validar que el objeto no fue alterado por el cliente.

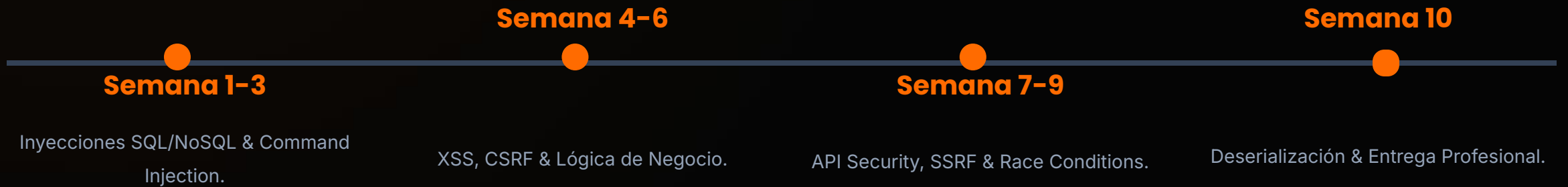
| Errores Comunes en Reportes

- ✗ **Copiar y Pegar:** No uses descripciones genéricas de scanners.
- ✗ **Falta de Contexto:** No explicar por qué ese bug importa para ese cliente específico.
- ✗ **Mala Ortografía:** Un reporte con errores mata tu credibilidad profesional.
- ✗ **Ser Alarmista:** Reportar un "Crítico" que en realidad no tiene impacto real.

| Checklist de Entregables

Elemento	Descripción
Reporte Principal (PDF)	Documento formal con hallazgos y mitigaciones.
Archivo de Evidencia	Logs, scripts de exploit o capturas de tráfico.
Resumen de Datos	CSV/Excel con la lista de vulnerabilidades para importación.
Presentación de Cierre	Resumen verbal de los puntos más críticos.

| Recorrido Hacker Web



CERTIFICACIÓN & CIERRE DEL TALLER

| Certificación QBit

Proceso de Finalización

Para obtener el certificado de Hacking web, deberás completar el examen integrador, y con base en la resolución de un laboratorio entregar un reporte técnico correspondiente antes de la fecha límite.

¡Revisa tu correo para las instrucciones finales!

¡GRACIAS POR EL VIAJE!

Este es solo el comienzo de tu carrera como Pentester.